

Python 基础速览 ¹

本指南旨在快速梳理 Python 最基础的知识点，帮助你在 CTF 解题或者其他使用时候时更加得心应手。

安装 Python

官方下载页面 (Windows): <https://www.python.org/downloads/windows/>

请选择 **Stable Releases** 版本，安装时务必勾选 **Add Python to PATH**

基础语法

一切的开始

```
print("Hello World")
```

基本数据类型与运算

赋值与类型

赋值，右值赋到左值

```
a = 1          # int
b = 1.0        # float
c = "abc"      # string
d = False      # boolean
```

算术运算

```
print(1 + 1)      # 2
print(1 + 1.0)    # 2.0
# print(1 + "1")  # TypeError
# print("1" + 1)  # TypeError
print("1" + "1")  # "11"

print(2 + 1)      # 3
print(2 - 1)      # 1
print(2 * 1)      # 2
print(2 / 1)      # 2.0
print(2 % 1)      # 0
print(2 ** 1)     # 2
print(3 // 2)     # 1 (整除)
```

比较运算

```
print(1 > 2)      # False
print(1 < 2)      # True
print(1 == 2)     # False
print(1 >= 2)     # False
```

复合赋值

```
a = 1
a += 1
a -= 1
a *= 2
a /= 2
```

位运算符不讲

逻辑运算符 AND OR NOT

成员运算符 in, not in

字符串格式化

```
name = "Starwalking"
age = 19

print(f"My name is {name}, I'm {age} years old")          # f-string
print("My name is {}, I'm {} years old".format(name, age)) # format()
```

流程控制

if-elif-else

```
if (1 == 1):
    pass    # 执行这里
elif (2 == 1):
    pass    # 不会进入
else:
    pass    # 也不会进入
```

for 循环

```
for i in range(10):
    print(i)    # 输出 0 到 9
```

while 循环

```
while(1):
    pass    # 无限循环
# 此处代码不会被执行（因为上面是死循环）
```

复合数据类型及常用方法

List（列表）

```
a = [1, 2, 3, 4, 5]
a.append(6)      # 尾部添加 6
a.pop()          # 删除最后一个元素 (6)
print(a[2])      # 输出第三个元素 (索引从 0 开始)
```

Tuple (元组) —不可变

```
a = (1, 2, 3)
# a.append(4)      # 错误！元组不可修改
b = (4,)
c = a + b          # 可以拼接两个元组
print(a[1])        # 输出第二个元素 (索引从 0 开始)
```

Set (集合) -用的不是很多

```
a = {1, 2, 3, 4}          # 直接定义
b = set([1, 2, 3, 4])     # 从列表创建
c = set()                 # 创建空集合 (注意不能使用 {})

print(1 in a)             # True
print(6 in a)             # False

d = a - b                 # 差集
d = a | b                 # 并集
d = a & b                 # 交集
d = a ^ b                 # 对称差
```

Dict (字典) —这个在网络请求处理json的时候用的比较多

```
a = {"key": "info"}
print(a["key"])           # 输出 "info"
```

函数

```
def add(a: int, b: int) -> int:
    return a + b
```

类、对象、Pickle这些会在Web题当中出现，此处不讲。

依赖管理

Python 拥有丰富的第三方库。

导入库

```
import requests
import numpy as pd    # 玩笑：把 numpy 重命名为 pd（实际 pandas 通常为 pd）
```

安装依赖

```
pip install requests
pip install -r requirements.txt
```

requests & flask

requests —— 发送 HTTP 请求

```
import requests

# GET 请求
a = requests.get("http://github.com/")
print(a.ok)    # 是否成功

# POST 请求（携带 JSON 数据）
data = {"something": "somedata"}
b = requests.post("http://github.com/", json=data)
print(b.text)    # 响应内容
```

关于 HTTP 请求类型的更多内容，请参考 Web 方向入门指南

flask — 轻量级后端框架

```
from flask import Flask
app = Flask(__name__)

@app.get("/")
def index():
    return "Hello"

if __name__ == "__main__":
    app.run()
```

Footnotes

1. 本文来自moectf [3-2 Python环境配置与基础语法](#) [↩](#)